

Trabalho 1 — MPI

Computação de Alto Desempenho

Hugo Leandro Antunes

DRE: 123143543

19 de junho de 2026

1 Exercício 1 — Variantes do Hello World

1.1 Exercício 1(a) — Introspecção em nível de SO

Abordagem: Cada processo MPI reporta, além do seu rank e número total de processos, o PID do processo no sistema operacional (via `getpid()`) e o núcleo de CPU em que está executando (via `sched_getcpu()`).

Compilação e execução:

```
mpicc -O2 -Wall -o hello_os hello_os.c
mpiexec -n 8 ./hello_os
```

Saída de 3 execuções consecutivas:

```
>> Execução 1:
Hello from rank 0/8 -- PID = 739, CPU = 10
...
Hello from rank 7/8 -- PID = 755, CPU = 6

>> Execução 2:
Hello from rank 0/8 -- PID = 766, CPU = 14
...
Hello from rank 7/8 -- PID = 783, CPU = 10

... (execuções 3 a 19 omitidas por brevidade) ...

>> Execução 20:
Hello from rank 0/8 -- PID = 904, CPU = 8
Hello from rank 1/8 -- PID = 905, CPU = 4
Hello from rank 2/8 -- PID = 906, CPU = 11
Hello from rank 3/8 -- PID = 910, CPU = 3
Hello from rank 4/8 -- PID = 912, CPU = 0
Hello from rank 5/8 -- PID = 915, CPU = 14
Hello from rank 6/8 -- PID = 919, CPU = 6
Hello from rank 7/8 -- PID = 921, CPU = 7
```

Observações:

1. **PIDs distintos:** Em cada execução, todos os 8 ranks possuem PIDs diferentes, confirmando que cada rank é um processo real do SO que roda concorrentemente.
2. **CPUs variando:** A coluna de CPU muda entre execuções. Isso ocorre porque o escalonador do kernel do Linux decide em qual núcleo cada processo é executado, e essa decisão é não-determinística — depende da carga do sistema no momento.

1.2 Exercício 1(b) — Variante em anel

Abordagem: Cada rank k envia sua saudação ao rank $(k+1) \bmod p$ e recebe do rank $(k-1+p) \bmod p$, formando um anel.

Escolha de implementação — MPI_Sendrecv: Optei por `MPI_Sendrecv` em vez de reordenar `Send/Recv` manualmente. A justificativa é que `MPI_Sendrecv` é uma operação atômica que realiza envio e recebimento simultaneamente, eliminando completamente o risco de deadlock circular. Um anel ingênuo com `Recv-antes-de-Send` gera deadlock porque todos os ranks ficam bloqueados esperando receber do vizinho à esquerda antes de enviar ao vizinho à direita.

O operador $+p$ em $(k-1+p) \bmod p$ garante argumento não-negativo para `%` em C, que retorna o sinal do dividendo.

Compilação e execução:

```
mpicc -O2 -Wall -o hello_ring hello_ring.c
mpiexec -n 4 ./hello_ring
```

Saída:

```
Rank 0 received from Rank 3: 'Greetings from rank 3 of 4!'
Rank 1 received from Rank 0: 'Greetings from rank 0 of 4!'
Rank 2 received from Rank 1: 'Greetings from rank 1 of 4!'
Rank 3 received from Rank 2: 'Greetings from rank 2 of 4!'
```

2 Exercício 2 — Regra do Trapézio Generalizada

Compilação e execução (geral para as variantes do Exercício 2):

```
mpicc -O2 -Wall -o mpi_trap_generalized mpi_trap_generalized.c -lm
mpiexec -n 3 ./mpi_trap_generalized 0 3.14159265358979323846 10
```

2.1 Exercício 2(a) — Balanceamento de carga para n arbitrário

Abordagem: Parti do código `mpi_trap_input.c` da aula e substituí a divisão uniforme `local_n = n/p` pelo esquema com resto:

```
1 int rem      = n % comm_sz;
2 int local_n  = n / comm_sz + (my_rank < rem ? 1 : 0);
3 int offset   = my_rank * (n / comm_sz)
4             + (my_rank < rem ? my_rank : rem);
5 double local_a = a + offset * h;
6 double local_b = local_a + local_n * h;
```

Os primeiros $n \bmod p$ ranks recebem $\lceil n/p \rceil$ trapézios, e os demais recebem $\lfloor n/p \rfloor$. A função de integração é $f(x) = \sin(x)$.

Verificação com $p = 3$, $n = 10$, $\int_0^\pi \sin(x) dx = 2$:

Divisão: $10 \bmod 3 = 1$, então rank 0 fica com 4 trapézios, ranks 1 e 2 com 3 cada.

```
RESULT: p=3 n=10 integral=1.983523537509454e+00 Tp=0.000011
Exact = 2.000000000000000e+00, Error = 1.6476e-02, h = 3.1416e-01
```

2.2 Exercício 2(b) — Estudo de escalabilidade forte

Definição: T_s é o tempo de relógio do código MPI executado com $p = 1$ (inclui overhead de `MPI_Init`/`MPI_Finalize`). Fixei $n = 10^7$ e tabulei para $p \in \{1, 2, 4, 8\}$. Cada configuração foi executada 11 vezes e o valor reportado é a **mediana**, eliminando outliers causados por variações do escalonador do SO.

Tabela 1: Escalabilidade forte — $n = 10^7$, $f(x) = \sin(x)$ em $[0, \pi]$ (mediana de 11 execuções).

p	T_p (s)	$S_p = T_1/T_p$	$E_p = S_p/p$
1	0.065372	1.0000	1.0000
2	0.034137	1.9150	0.9575
4	0.017820	3.6685	0.9171
8	0.011797	5.5414	0.6927

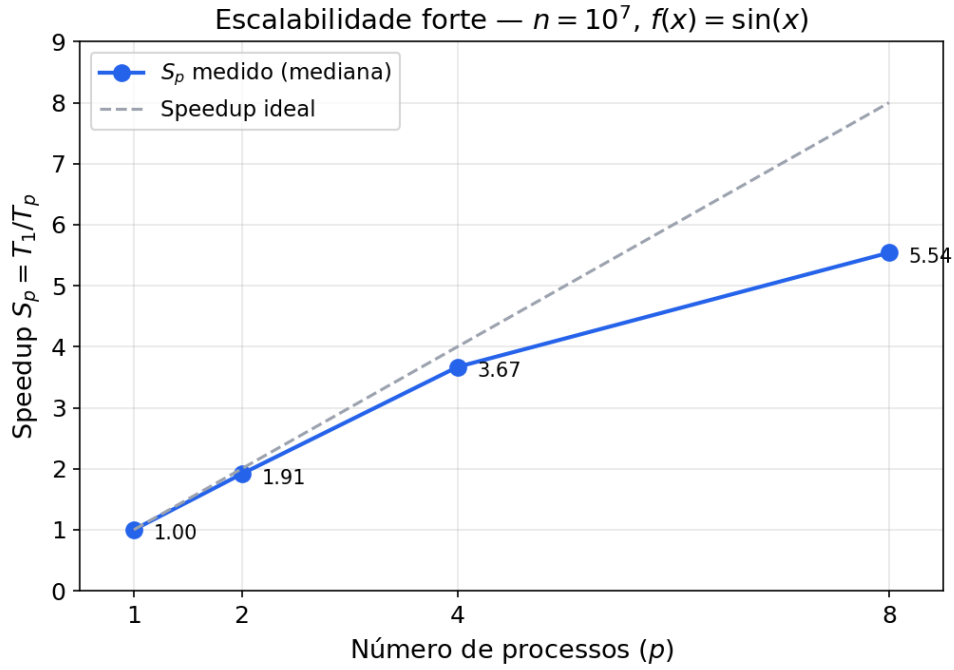


Figura 1: Speedup S_p vs. número de processos p (escalabilidade forte, $n = 10^7$).

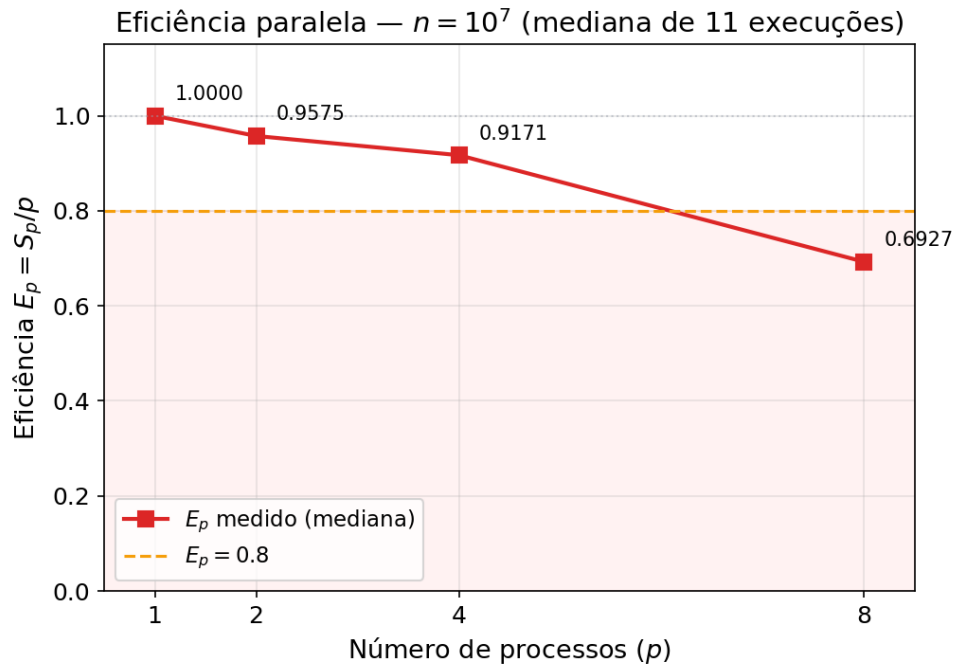


Figura 2: Eficiência E_p vs. número de processos p ($n = 10^7$, mediana de 11 execuções).

Discussão: A eficiência E_p permanece acima de 0.9 para $p \leq 4$, mas cai para ≈ 0.69 em $p = 8$. A máquina de teste possui 8 núcleos físicos (AMD Ryzen 7 8700G), portanto não há oversubscription em $p = 8$. A queda de eficiência se explica pela granularidade fina do trabalho: com $n/p = 1,25 \times 10^6$ trapézios por processo e tempo total ~ 12 ms, a latência fixa de comunicação (MPI_Reduce) e sincronização (MPI_Barrier) domina proporcionalmente mais o tempo total. A eficiência cai abaixo de 0.8 a partir de $p = 8$.

2.3 Exercício 2(c) — Bônus: Convergência $O(h^2)$

Fixei $p = 4$ e variei $n \in \{64, 256, 1024, 4096, 16384\}$.

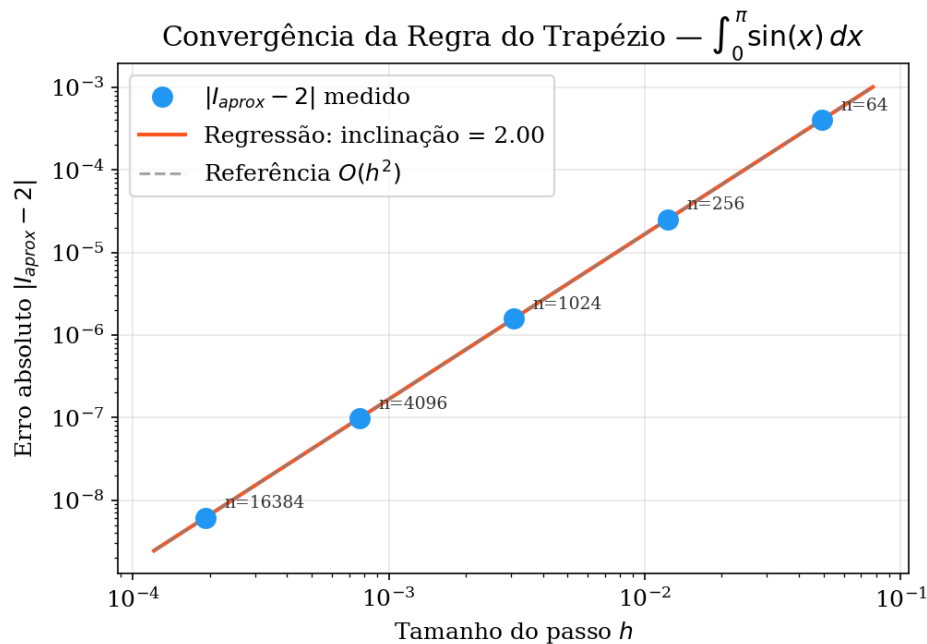


Figura 3: Convergência log-log: $|\text{erro}|$ vs. h para a regra do trapézio.

Resultado: A inclinação da reta de regressão em escala log-log é ≈ 2.0 , confirmando a taxa de convergência $O(h^2)$ da regra do trapézio composta.

3 Exercício 3 — Soma Paralela

Abordagem:

1. Rank 0 gera vetor de $N = 10^7$ doubles aleatórios (seed fixa = 42).
2. Rank 0 distribui N/p elementos a cada rank via laço de `MPI_Send`.
3. Cada rank calcula soma local.
4. Cada rank envia soma parcial ao rank 0 via `MPI_Send`.
5. Rank 0 soma as parciais e compara com soma serial.

Compilação e execução:

```
mpicc -O2 -Wall -o mpi_psum mpi_psum.c -lm
mpiexec -n 4 ./mpi_psum 10000000
```

Saída:

```
N = 10000000, p = 4
Soma serial = 4.999361300896658e+06
Soma paralela = 4.999361300896985e+06
Erro relativo = 6.5573e-14
```

Análise: O erro relativo é da ordem de 10^{-14} , decorrente da reordenação de ponto flutuante — a ordem das somas parciais difere entre as versões serial e paralela, e o acúmulo de erros de arredondamento ao longo de $N = 10^7$ operações amplifica a diferença para além da precisão de máquina unitária ($\varepsilon \approx 10^{-16}$).

Comparação com versão coletiva (Aula 6): Na versão com coletivas, o pipeline `MPI_Send + MPI_Recv` é substituído por `MPI_Scatter` (distribuição) e `MPI_Reduce` (redução). Isso reduz significativamente a quantidade de código:

Tabela 2: Comparação de linhas de código — soma paralela.

Versão	Linhas de código
Ponto-a-ponto (<code>Send/Recv</code>)	~90
Coletivas (<code>Scatter/Reduce</code>)	~40

4 Exercício 4 — Prática de Comunicação Coletiva

4.1 Exercício 4(a) — Hello com `MPI_Gather`

Abordagem: Cada rank prepara uma saudação em um buffer de tamanho fixo (`MAX_STRING = 100`). O rank 0 usa `MPI_Gather` para coletar todas as saudações em um vetor contíguo, substituindo o laço de `MPI_Recv` do código original.

Compilação e execução:

```
mpicc -O2 -Wall -o hello_gather hello_gather.c
mpiexec -n 4 ./hello_gather
```

Saída:

```
Greetings from process 0 of 4!
Greetings from process 1 of 4!
Greetings from process 2 of 4!
Greetings from process 3 of 4!
```

4.2 Exercício 4(b) — Máx/mín global com `MPI_Reduce`

Abordagem: Rank 0 gera $N = 10^6$ doubles aleatórios em $[-500, 500]$. `MPI_Scatter` distribui blocos iguais. Cada rank calcula máx e mín locais. `MPI_Reduce` com `MPI_MAX` e `MPI_MIN` produz os extremos globais. Rank 0 verifica contra percurso serial.

Compilação e execução:

```
mpicc -O2 -Wall -o minmax minmax.c -lm
mpiexec -n 4 ./minmax 1000000
```

Saída:

```
N = 1000000, p = 4
MPI_Reduce MAX = 499.999, MIN = -499.998
Serial MAX = 499.999, MIN = -499.998
Verificação: MAX OK, MIN OK
```

5 Exercício 5 — Soma Paralela de Vetores

Abordagem: Rank 0 inicializa $x_i = i$ e $y_i = 2i$. `MPI_Scatter` distribui blocos de x e y . Cada rank computa $z_{\text{local}} = x_{\text{local}} + y_{\text{local}}$.

Compilação e execução:

```
mpicc -O2 -Wall -o mpi_vecadd_gather mpi_vecadd_gather.c
mpiexec -n 4 ./mpi_vecadd_gather 16

mpicc -O2 -Wall -o mpi_vecadd_allgather mpi_vecadd_allgather.c
mpiexec -n 4 ./mpi_vecadd_allgather 16
```

5.1 Versão com MPI_Gather

Rank 0 coleta z via `MPI_Gather` e verifica $z_i = 3i$.

5.2 Versão com MPI_Allgather

Todos os ranks recebem o vetor z completo via `MPI_Allgather`. Cada rank verifica localmente.

Saídas:

```
=== Gather ===
z = x + y (Gather):
  z[ 0] = 0   (expected 0)
  z[ 1] = 3   (expected 3)
  ...
  z[15] = 45  (expected 45)
Verificação: OK

=== Allgather ===
[Rank 0] z = x + y (Allgather):
  z[ 0] = 0
  ...
  z[15] = 45
Verificação: OK
[Rank 1] z = x + y (Allgather): ...
[Rank 2] z = x + y (Allgather): ...
[Rank 3] z = x + y (Allgather): ...
```

Discussão — Quando MPI_Allgather é preferível:

`MPI_Allgather` é útil quando *todos* os processos precisam do resultado completo para computação subsequente. Exemplos:

- **Multiplicação matriz-vetor iterativa:** Em métodos como Gradiente Conjugado, cada rank calcula uma parte de $y = Ax$, mas todos precisam do vetor y completo para a próxima iteração.
- **Busca global:** Quando cada processo precisa saber o estado global para tomar decisões locais (ex: critérios de convergência).

A desvantagem é o custo de comunicação: `MPI_Allgather` envia $O(N)$ dados a *todos* os ranks, enquanto `MPI_Gather` envia apenas ao root.

6 Exercício 6 — Tipo de Dado Derivado MPI

Struct:

```
1 struct Student {
2     char    name[50];
3     double  grade;
4     int     id;
5 };
```

Compilação e execução:

```
mpicc -O2 -Wall -o student_struct student_struct.c
mpiexec -n 4 ./student_struct

mpicc -O2 -Wall -o student_three_bcasts student_three_bcasts.c
mpiexec -n 4 ./student_three_bcasts
```

6.1 Versão com tipo derivado

Construí um tipo MPI com `MPI_Type_create_struct` usando `offsetof` de `<stddef.h>` para deslocamentos corretos (que respeitam o padding inserido pelo compilador). Uma única chamada `MPI_Bcast` transmite o registro completo.

6.2 Versão com três MPI_Bcast

Três chamadas separadas: uma para **name** (50 chars), uma para **grade** (1 double), uma para **id** (1 int).

Comparação:

Tabela 3: Comparação entre tipo derivado e três Bcasts.

Versão	Chamadas MPI_Bcast	Conteúdo idêntico?
Tipo derivado	1	Sim
Três Bcasts	3	Sim

O conteúdo recebido é **idêntico** em ambas versões. A vantagem do tipo derivado é:

1. **Eficiência:** Menos chamadas MPI reduz latência acumulada (cada **Bcast** incorre em overhead de sincronização $O(\log p)$).
2. **Escalabilidade:** Para structs com muitos campos, o número de chamadas com a abordagem de Bcasts separados cresce linearmente; com tipo derivado, permanece 1.
3. **Manutenabilidade:** Adicionar um campo à struct requer apenas atualizar a definição do tipo derivado, em vez de lembrar de adicionar mais uma chamada **Bcast**.

Saídas:

```
=== Tipo derivado (1 Bcast) ===
[Rank 0] Enviando Student via tipo derivado (1 MPI_Bcast):
  name = "Hugo Leandro Antunes"
  grade = 9.5
  id = 123143543
[Rank 1] Recebido via tipo derivado:
  name = "Hugo Leandro Antunes"
  grade = 9.5
  id = 123143543
[Rank 2] Recebido via tipo derivado:
  name = "Hugo Leandro Antunes"
  grade = 9.5
  id = 123143543
[Rank 3] Recebido via tipo derivado:
  name = "Hugo Leandro Antunes"
  grade = 9.5
  id = 123143543

=== Tres Bcasts separados ===
[Rank 0] Enviando Student via 3 MPI_Bcast separados:
  name = "Hugo Leandro Antunes"
  grade = 9.5
  id = 123143543
[Rank 1] Recebido via 3 Bcasts:
  name = "Hugo Leandro Antunes"
  grade = 9.5
  id = 123143543
[Rank 2] Recebido via 3 Bcasts:
  name = "Hugo Leandro Antunes"
  grade = 9.5
  id = 123143543
[Rank 3] Recebido via 3 Bcasts:
  name = "Hugo Leandro Antunes"
  grade = 9.5
  id = 123143543
```